

# Chapter 8

## Hashing

---

### 8.1 Introduction

We first consider the ADT **Dictionary** (ADT 5.3) again.

## ADT 5.3

```
template <class K, class E>
```

```
class Dictionary {
```

```
public:
```

```
    virtual bool IsEmpty () const = 0;
```

```
    // return true iff the dictionary is empty
```

```
    virtual pair<K,E>* Get(const K&) const = 0;
```

```
    // return pointer to the pair with specified key;
```

```
    // return 0 if no such pair
```

```
    virtual void Insert(const pair<K,E>&) = 0;
```

```
    // insert the given pair; if key is a duplicate
```

```
    // update associated element
```

```
    virtual void Delete(const K&) = 0;
```

```
    // delete pair with specified key
```

```
};
```

## ■ The 3 basic operations on Dictionary are **Get**, **Insert** and **Delete**.

- When a dictionary is represented as a **binary search tree**, these operations take  **$O(h)$**  time, where  $h$  is the height of the tree.
- In this chapter, we introduce a technique, called **hashing**, which enables us to perform these operations in  **$O(1)$**  if a dictionary is represented as a hash table.

## 8.2 Static Hashing

### Definitions:

- In static hashing, the dictionary pairs are stored in a table,  $ht$ , called hash table.
- The hash table  $ht$  is partitioned into  $b$  buckets:  $ht[0], \dots, ht[b-1]$ . Each bucket holds  $s$  slots, each slot holds exactly one pair. Usually,  $s = 1$ .
- We use a function called the hash function  $h$  to map a dictionary pair's key into a bucket.
- To be specific, when the dictionary pair's key is  $k$ , we can compute  $h(k) \in \{0, 1, \dots, b-1\}$ .  $h(k)$  is called the hash address (home address) of  $k$ . According to  $h(k)$ , we can determine the storage address (location) of the corresponding dictionary pair.

- **key density**---The key density of a hash table is the ratio  $n/T$ , where  $n$  is the number of pairs in the hash table and  $T$  is the total number of possible keys.
- **loading density (or loading factor)** ---The loading density of a hash table is defined by  $\alpha = n/(sb)$ .
- Usually  $b \ll T$ . Therefore, the hash function  $h$  maps several different keys into the same bucket.
- **synonyms**---2 keys  $k_1$  and  $k_2$  are said to be synonyms with respect to  $h$  if  $h(k_1) = h(k_2)$ .

- collision---a collision occurs when the home bucket for the new pair is **not empty** at the time of insertion.
- overflow---an overflow occurs when home bucket for the new pair is **full** at the time we wish to insert this pair into the table.
- when  $s=1$ , collisions and overflows occur simultaneously.

## Example 8.1:

Consider a hash table with  $b=26$  buckets and  $s=2$ . There are  $n=10$  distinct keys (each key begins with a letter) : GA, D, A, G, L, A2, A1, A3, A4, and E. The hash function is:  $h(k)$  = the first character of  $k$  - 'A'.

So, the hash addresses are: 6, 3, 0, 6, 11, 0, 0, 0, 0, and 4.

The keys GA and G are **synonyms**.

The keys A, A2, A1, A3 and A4 are **synonyms**.

The **loading factor**  $\alpha = 10/(26*2) = 0.19$ .

The keys GA, D, A, G, L, A2, A1 entering into the hash table are shown as follows:

The order of keys entering into the hash table is:  
**GA, D, A, G, L, A2, A1.**

| ht | Slot 1 | Slot 2 |
|----|--------|--------|
| 0  | A      | A2     |
| 1  |        |        |
| 2  |        |        |
| 3  | D      |        |
| 4  |        |        |
| 5  |        |        |
| 6  | GA     | G      |
| .  | .      | .      |
| .  | .      | .      |
| 25 |        |        |

**L is in:  
slot 1, bucket 11**

**When A1 hashes into ht[0] --- overflow.**



## we can see:

- If **no overflow occurs**, the time required to insert, delete or search using hash depends only on **the time for computing the hash function** and the time for searching one bucket.
  - ✓ These times are independent of  $n$ .
- We would like to choose a hash function that is **both easy to compute and result in very few collisions**.
- But  $T \gg b$  and  $s$  is small, it is **impossible to avoid overflow**.
  - ✓ A mechanism to **handle overflows** is necessary.

## 8.2.2 Hash Functions

- A **hash function** maps a key into a **bucket**. It is desired that the hash function is **easy to compute** and **minimize the number of collisions**.
- A hash function satisfying the following property is a **uniform hash function**: if  $k$  is a key chosen at **random** from the key space, then the probability that  $h(k)$  equals to  $i$  is  **$1/b$**  for all buckets  $i$ .

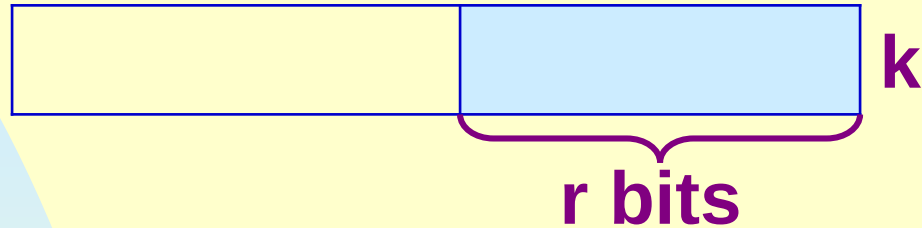
■

## 8.2.2.1 Division

- keys are non-negative integers.
- $h(k) = k \% D$
- The hash table must have **at least D buckets**.
- The **number of overflows** on real dictionaries is critically dependent on **the choice of D**.

For instance:

(1) If  $D=2^r$ , then  $h(k)$  depends only on the last  $r$  bits of  $k$ :

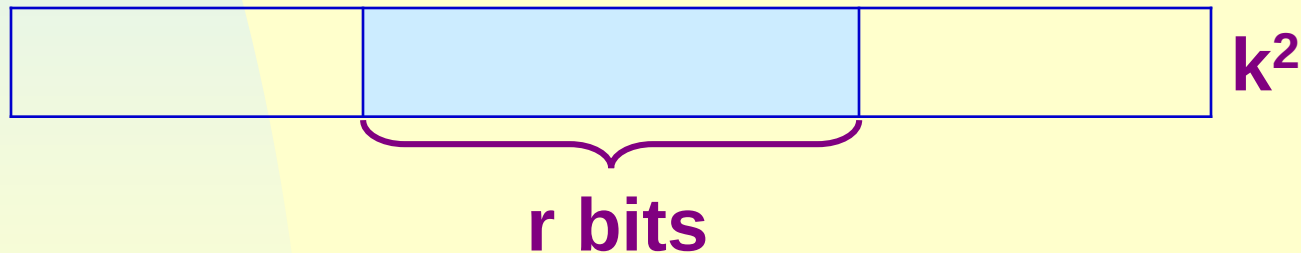


(2) If  $D$  is divisible by 2, then odd keys are mapped to odd buckets, even keys to even buckets.

In practice, we choose  $D$  such that it has **no prime factor smaller than 20**.

## 8.2.2.2 Mid-Square

- The key is assumed to be an integer.
- $h(k)$  is computed by using an appropriate number of bits from the middle of  $k^2$  to obtain the bucket address.
- If  $r$  bits used, then  $b = 2^r$ .



### 8.2.2.3 Folding

- $k$  is partitioned into several parts, all but possibly the last being of the same length.
- These partitions are then added together to obtain the hash address for  $k$ .
- There are two folding methods:
  - ✓ shift folding
  - ✓ folding at the boundaries

## Example 8.2:

$k=12320324111220$  is partitioned into parts that are 3 decimal digits long.

$P_1=123$ ,  $P_2=203$ ,  $P_3=241$ ,  $P_4=112$ ,  $P_5=20$ .

- **shift folding**

$$h(k)=123+203+241+112+20=699$$

- **folding at the boundaries**

$$h(x)=123+302+241+211+20=897$$

## 8.2.2.4 Digit Analysis

- Particularly useful in the case of a **static file** where **all the keys in the table are known in advance**.
- Each  $k$  is **interpreted as a number** based on some radix  $r$ .
- Digits **having the most skewed distribution are deleted**.
- When enough digits are deleted, the number of remaining digits is small enough to obtain an address in the range of the hash table.



For example, consider the students' identities:

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 7 | 1 | 1 | 2 | 4 | 1 | 0 | 1 |
| 7 | 1 | 1 | 2 | 4 | 1 | 0 | 2 |
| 7 | 1 | 1 | 2 | 4 | 1 | 0 | 3 |
|   |   |   |   |   | . |   |   |
|   |   |   |   |   | . |   |   |
| 7 | 1 | 1 | 2 | 4 | 4 | 2 | 7 |
| 7 | 1 | 1 | 2 | 4 | 4 | 2 | 8 |
| 7 | 1 | 1 | 2 | 4 | 4 | 2 | 9 |

Let  $b=1000$ , delete digits “71124”, left the last 3 digits as the bucket address.

## 8.2.2.5 Converting Keys to integers

- To use some of the described hash functions, **keys need to first be converted to nonnegative integers.**
- It is **not necessary** to convert keys into **unique nonnegative integers**, because **conflict is inevitable.**
- For example, it is OK for us to convert the strings “data”, “structures”, and “algorithms” **into the same integer** (say 199).

## Example 8.3: Converting Strings to Integers.

- Since it is **not necessary to convert strings into unique nonnegative integers**. We can map every string, no matter how long, into a **16-bits integers**:

```
unsigned int StringToInt(string s)
{ // Convert s into a nonnegative int that depends on all
  // characters in s
  int length = (int) s.length(); // number of characters in s
  unsigned int answer = 0;
  if (length%2 == 1) { // length is odd
    answer = s.at(length-1);
    length--;
  }
```

```
// length is now even
for (int i=0; i<length; i+=2)
{ // do 2 characters at a time
    answer += s.at(i);
    answer += ((int) s.at(i+1))<<8;
}
return answer;
}
```

## 8.2.4 Overflow Handling: open addressing and chaining

### 8.2.4.1 Open Addressing

- linear probing (linear open addressing)
- quadratic probing
- rehashing
- random probing

## Linear probing

- When inserting a new pair whose key is  $k$ , we search the hash table buckets in the order  $h(k)$ ,  $h(k)+1\%b, \dots, h(k)+b-1\%b$ .
- The search terminates when we reach **the first unfilled bucket** and the new pair is inserted into this bucket.
- In case no such bucket is found, the hash table is full and it is necessary to increase the table size.
- In practice, to ensure good performance, table size is increased when the loading density exceed a prespecified threshold such as 0.75 rather than the table is full.

- When we resize the hash table, we must change the hash function as well.
- This change in the hash function potentially change the home bucket for each key in the hash table. So all dictionary entries need to remapped into the new larger table.

### Example 8.6:

- $b=26$ ,  $s=1$ .
- keys: GA, D, A, G, L, A2, A1, A3, A4, Z, ZA, E.
- $h(k)$ =first character of  $k$  - 'A'.
- Initially, the hash table  $ht$  is empty.

|    |    |
|----|----|
| 0  | A  |
| 1  | A2 |
| 2  | A1 |
| 3  | D  |
| 4  | A3 |
| 5  | A4 |
| 6  | GA |
| 7  | G  |
| 8  | ZA |
| 9  | E  |
| 10 |    |
| 11 | L  |
| .  | .  |
| .  | .  |
| 24 |    |
| 25 | Z  |

After entering

GA, D, A, G, L, A2, A1, A3,  
A4, Z, ZA, E,

we get the table shown  
left.



# Search algorithm when **linear probing** is used to handle overflows

- When  $s=1$  and **linear probing** is used to handle overflows, **search for key  $k$**  proceeds as follows:
  - (1) Compute  $h(k)$
  - (2) Examine  $ht[h(k)]$ ,  $ht[h(k)+1\%b]$ , ...,  $ht[h(k)+j\%b]$  **until** one of the following happens:
    - (a)  $ht[h(k)+j\%b]$  has a pair whose key is  $k$ --- **found.**
    - (b)  $ht[h(x)+j\%]$  is empty---  **$k$  is not in the table.**
    - (c) return to the starting position  $ht[h(k)]$ --- **the table is full and  $k$  is not in the table.**

# Class definition of LinearProbing

The following are the **class definitions**:

```
template<class K, class E>
typedef pair<K, E> * pairPtr;
class LinearProbing {
public:
    LinearProbing (int size) {
        if (size <3) throw "the size must be >= 3.";
        b=size;
        ht=new pairPtr[b]; //ht stores pointers to dictionary pairs
        fill(ht, ht+b,0); // initialize the hash table to empty.
    };
private:
    int b;
    pairPtr *ht;
    //assume the hash table ht sores pointers to dictionary pairs
};
```

## Program 8.4: Get function of linear probing

```
template <class K, class E>
pair<K, E>* LinearProbing<K, E>::Get(const K& k)
{ // search the linear probing hash table ht (s=1) for k.
  // return a pointer to the found pair or 0.
  int i=h(k); // home bucket
  int j;
  for (j=i; ht[j] && ht[j]→first !=k;) {
    j = (j+1) % b; // treat the table as circular
    if ( j == i ) return 0; // back to the start point
  }
  if (ht[j] && ht[j]→first==k) return ht[j];
  return 0; //ht[j]==0
}
```

## Performance of linear probing

- When linear probing is used to resolve overflows, keys **tend to cluster together**. Moreover, adjacent clusters tend to coalesce(合并), thus increasing the search time.
- When linear probing is used together with a uniform hash function, the expected **average number of key comparisons** to look up a key is approximately  $(2 - \alpha) / (2 - 2\alpha)$ , where  $\alpha$  is the loading density.
- In Example 8.6,  $\alpha = 12 / 26 = 0.47$  and the average key comparisons is 1.5.
- Although the average number of comparisons is **small**, the worst case can be quite **large**.

## Quadratic probing

- A quadratic function of  $i$  is used as the search increment. In particular, the search is carried out by examining buckets  $h(k)$ ,  $(h(k)+1)\%b$ ,  $(h(k)-1)\%b$ ,  $(h(k)+2^2)\%b$ ,  $(h(k)-2^2)\%b$ ....
- When  $b$  is a prime number of the form  $4j+3$ , for  $j$  an integer, the quadratic search described above examines every bucket in the table.

## Rehashing

- Use a series of hash functions  $h_1, h_2, \dots, h_m$ .
- Buckets  $h_i(k)$ ,  $1 \leq i \leq m$  are examined in order.

## Random probing

- Examining the buckets in the order  $h(k)$ ,  $(h(k)+s(1))\%b$ ,  $(h(k)+s(2))\%b$ , ...  $(h(k)+s(b-1))\%b$
- $s(i)$  ( $1 \leq i \leq b-1$ ) is a pseudo random number. The random number generator must satisfy the property that every number from 1 to  $b-1$  must be generated exactly once as  $i$  ranges from 1 to  $b-1$ .

## 8.2.4.2 Chaining

- **Linear probing** and its variations performs poorly because when search a key, the keys **having different hash values are probably compared too**.
- Many of the comparisons can be saved if we maintain **lists of synonyms, one list per bucket**, each list containing all the **synonyms** for that bucket.
- If so, a search needs **computing the hash address  $h(k)$  and examining the keys in the list for  $h(k)$** .

- Although the list for  $h(k)$  may be maintained using any data structure that supports the search, insert and delete operations (e.g. arrays, chains, search trees, etc), **chains** are most frequently used.
- We typically use an array  $ht[0:b-1]$  of type **`ChainNode<pair<K,E>>*`** to express the hash table with  $ht[i]$  pointing to the first node of the chain for bucket  $i$ .



## Class definition of Chaining

```
template <class K, class E>
typedef ChianNode<pair<K, E>>* ChainNodePtr;
class Chaining { // assume Chining is friend of ChainNode
public:
    Chaining(int size) {
        if (size <3) throw "the size must be >= 3.";
        b=size;
        ht=new ChainNodePtr[b];
        fill(ht, ht+b,0); // initialize the hash table to empty.
    };
private:
    int b;
    ChainNodePtr *ht;
};
```

## Get function of Chaining

```
template <class K, class E>
```

```
Pair<K, E>* Chaining<K, E>::Get(const K& k)
```

```
{ // Search the chained hash table ht for k. If a pair with key k  
  // is found return a pointer to this pair, else return 0.
```

```
  int i = h(k); // home bucket
```

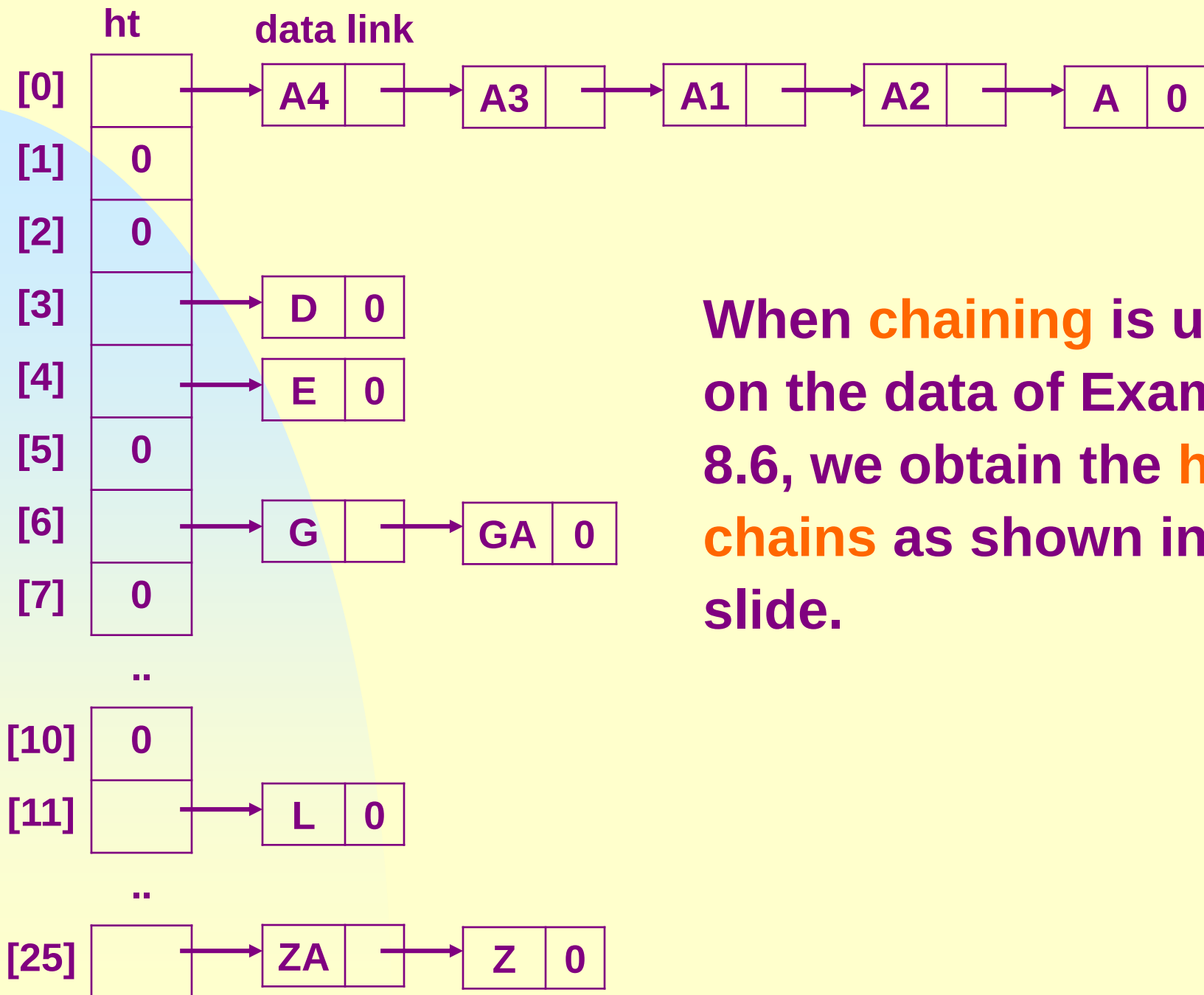
```
  // search the chain starting at ht[i]
```

```
  for (ChainNode<pair<K, E>>* current=ht[i]; current;  
                                              current=current→link)
```

```
    if (current→data.first==k) return &current→data;
```

```
  return 0;
```

```
}
```



When **chaining** is used on the data of Example 8.6, we obtain the **hash chains** as shown in this slide.

- When chaining is used along with a **uniform hash function**, the expected average number of key comparisons for a **successful search** is

$$\approx 1 + \alpha / 2$$

where  $\alpha$  is the **loading density**  $n/b$ .

# Comparison between Chaining (C) and Linear Probing (LP)

The number in each cell is the comparison times.

| $\alpha = n/b$ | 0.50 |       | 0.75 |       | 0.90 |       | 0.95 |        |
|----------------|------|-------|------|-------|------|-------|------|--------|
| Hash function  | C    | LP    | C    | LP    | C    | LP    | C    | LP     |
| mid square     | 1.26 | 1.73  | 1.40 | 9.75  | 1.45 | 37.14 | 1.47 | 37.53  |
| division       | 1.19 | 4.52  | 1.31 | 7.20  | 1.38 | 22.42 | 1.41 | 25.79  |
| shift fold     | 1.33 | 21.75 | 1.48 | 65.10 | 1.40 | 77.01 | 1.51 | 118.57 |
| bound fold     | 1.39 | 22.97 | 1.57 | 48.70 | 1.55 | 69.63 | 1.51 | 97.56  |
| digit analysis | 1.35 | 4.55  | 1.49 | 30.62 | 1.52 | 89.20 | 1.52 | 125.59 |
| theoretical    | 1.25 | 1.50  | 1.37 | 2.50  | 1.45 | 5.50  | 1.48 | 10.50  |

- Chaining outperforms linear open addressing for overflow handling
- Division is superior to other hash functions.

# **Exercise(s) and Experiment(s)**

## **Exercise(s):**

- **P475:3**

## **Experiment(s):**

- **P475:6**